

Spatial autocorrelation (introduction)

Jérôme Guélat, Swiss Ornithological Institute (2013)

Introduction

The first law of geography says: “Everything is related to everything else, but near things are more related than distant things” (Tobler, 1970). This phenomenon is called **spatial autocorrelation** by statisticians. It can be seen as a simple 2D generalization of temporal autocorrelation, which describes the tendency of two things nearer to each other along the time axis to be more similar than things further apart in time. Legendre (1993) proposed a more formal definition: “The property of random variables taking values, at pairs of locations a certain distance apart, that are more similar (positive autocorrelation) or less similar (negative autocorrelation) than expected for randomly associated pairs of random observations”. The two most common examples of positive autocorrelation in ecology are patchiness and gradients, and a regular distribution is an example of negative autocorrelation.

Spatial autocorrelation is a very general property of most ecological datasets, and can occur at all spatial scales. An observation in close proximity to another one does not substantially increase the information in the data because it is similar to the one already measured. The value of a random variable characterizing a site (e.g. a habitat covariate or the local density of an animal species) can thus be partially predicted by the values at neighbouring sites. Such measurements only artificially increase sample size without contributing a full unit of independent information. Autocorrelation can thus be described as one of the mechanisms leading to so-called pseudoreplication (Hurlbert, 1984) which will lead to an overestimation of the precision of the results (e.g. the confidence intervals will be too small).

In the case of species distributions, spatial autocorrelation occurs mostly because of habitat heterogeneity (e.g. a species which occurs only in forests) or because of biotic processes such as dispersal, conspecific attraction, competition with another species or other complex dynamics (e.g. source-sink). When modelling species distributions, the presence of spatial autocorrelation in the residuals is very often an indication that an important covariate was not included in the model (or that the model was misspecified in another way).

```
# Load the packages needed for this tutorial
library(raster)
```

```
## Loading required package: sp
```

```
## warning: package 'sp' was built under R version 3.0.2
```

```
library(gstat)
library(lattice)
```

```
## warning: package 'lattice' was built under R version 3.0.2
```

```
library(ncf)
set.seed(45)
```

Generation of spatially autocorrelated data

In order to better understand spatial autocorrelation, we will first learn how to simulate spatially autocorrelated data. We will model the variable using a multivariate normal distribution $(X \sim \text{MVN}(0, \Sigma))$, where the covariance matrix (Σ) incorporates the spatial association. A function (D) representing the decay in correlation between pairs of points with distance is used to compute (Σ) . The most commonly used model is the exponential one which model similarity between sites as a exponential decay with distance. If (δ_{ij}) represents the distance between points i and j , then $(D(\delta_{ij}) = e^{-\phi \delta_{ij}})$, where (ϕ) is the parameter describing how rapidly the correlation declines with distance.

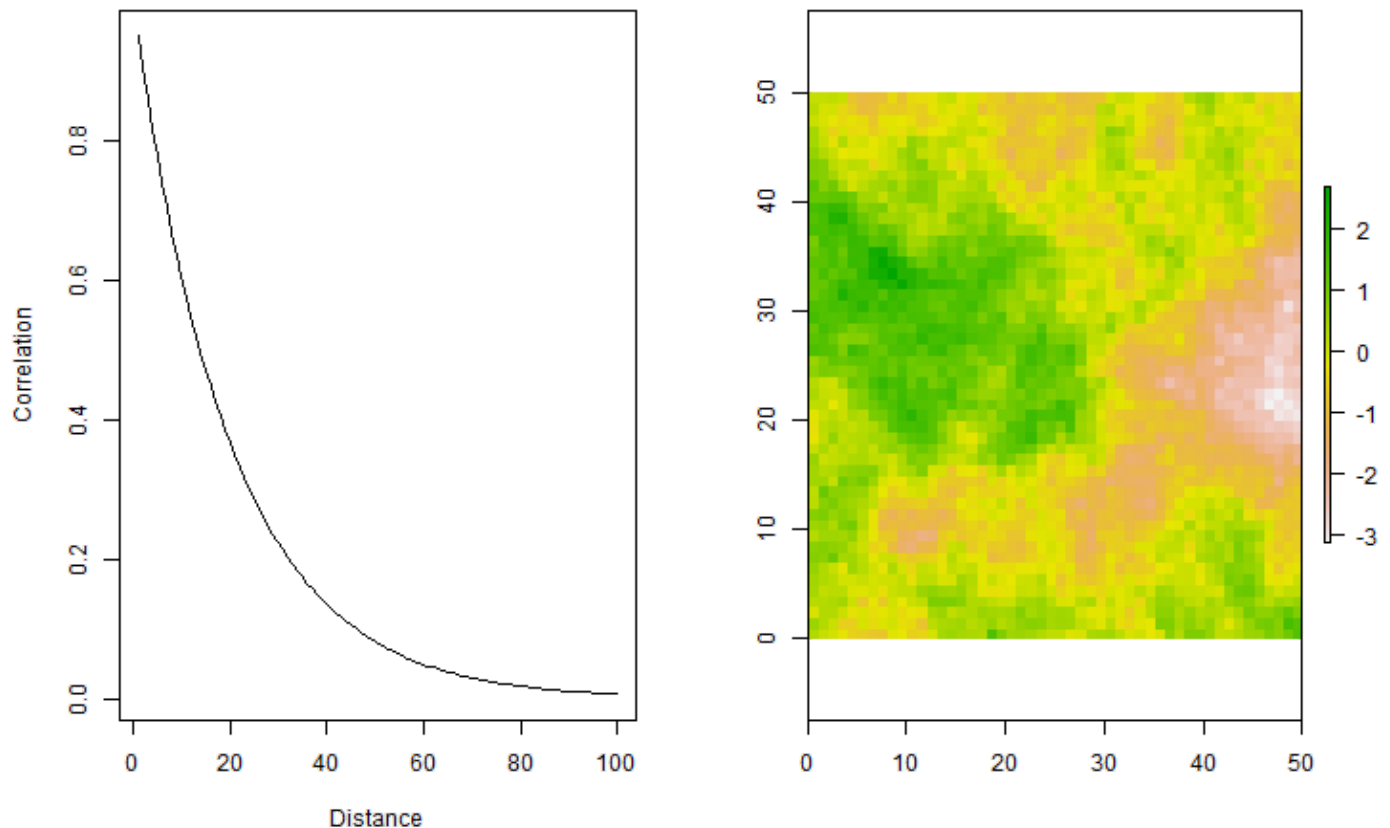
Please note that in the following, each one of you will generate a different dataset; hence, you cannot expect that your parameter estimates later, and your graphs and plots will look exactly like ours here. This is because the pseudo-random-number generators are initialised at different values. If you want to “fix” your random numbers, use the R command `set.seed(x)`, with x some number of your choice, e.g. 24. Executing the following commands repeatedly will then always result in exactly the same estimates and graphs.

```
# Define function to draw random samples from a multivariate
normal
# distribution
rmvn <- function(n, mu = 0, v = matrix(1)) {
  p <- length(mu)
  if (any(is.na(match(dim(v), p))))
    stop("Dimension problem!")
  D <- chol(v)
  t(matrix(rnorm(n * p), ncol = p) %*% D + rep(mu, rep(n, p)))
}

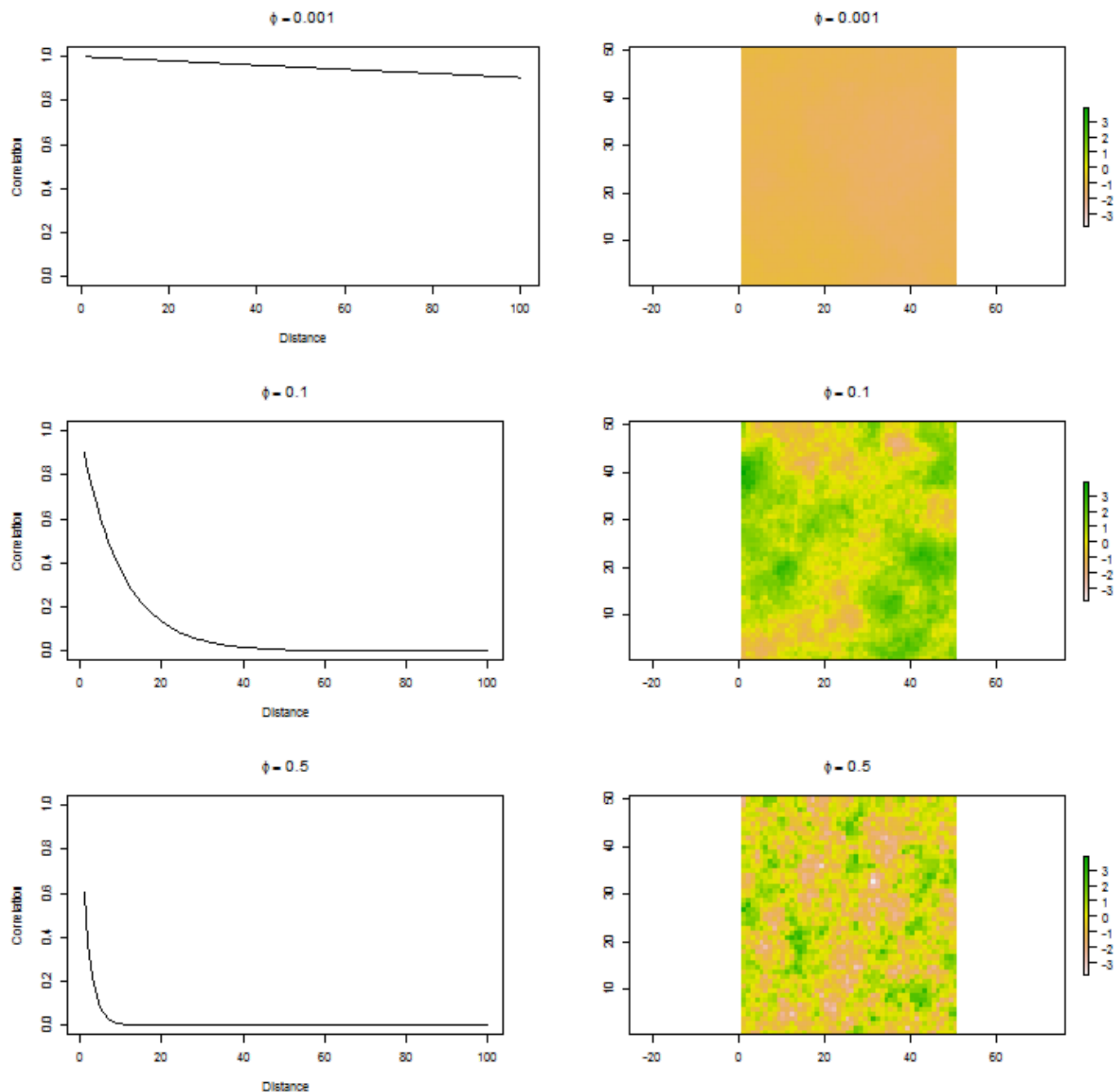
# Set up a square lattice region
simgrid <- expand.grid(1:50, 1:50)
n <- nrow(simgrid)

# Set up distance matrix
distance <- as.matrix(dist(simgrid))
# Generate random variable
phi <- 0.05
X <- rmvn(1, rep(0, n), exp(-phi * distance))

# visualize results
Xraster <- rasterFromXYZ(cbind(simgrid[, 1:2] - 0.5, X))
par(mfrow = c(1, 2))
plot(1:100, exp(-phi * 1:100), type = "l", xlab = "Distance", ylab =
"Correlation")
plot(Xraster)
```



If we run this code several times with different values of ϕ , we can generate different patterns and strengths of spatial autocorrelation. Indeed, it is a useful exercise to change the value of ϕ and observing how the “size” of the clusters and the values of the variable vary. Note that another correlation structure (e.g. gaussian or spherical) would produce other patterns.

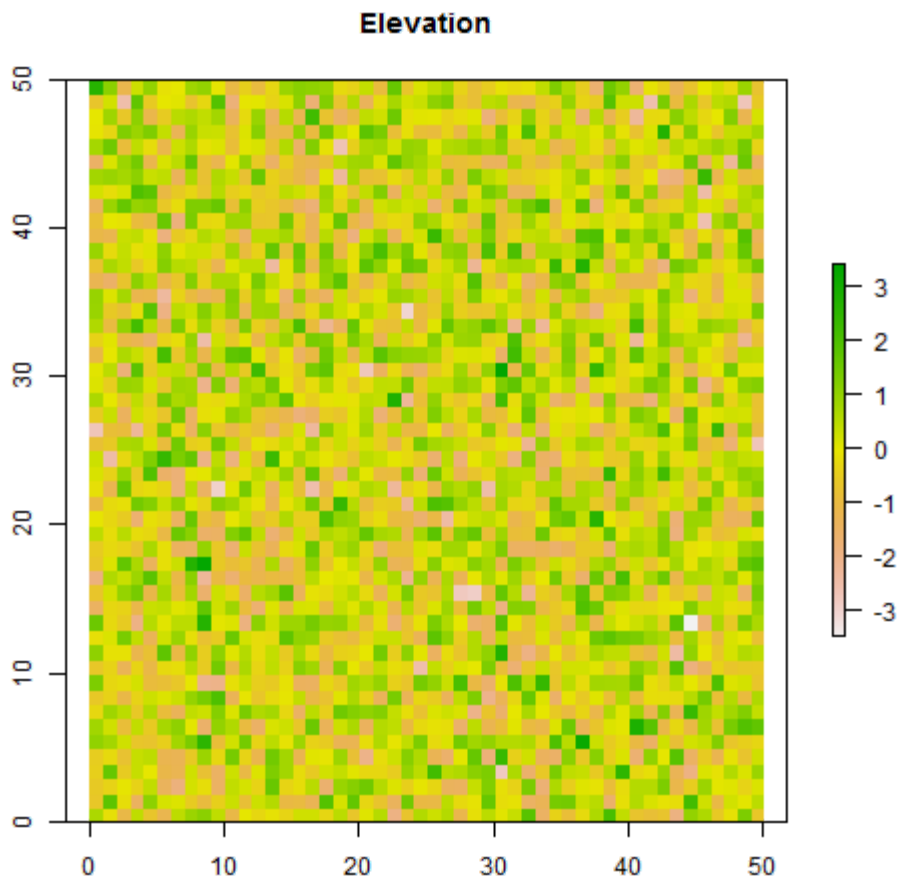


We note that higher values of ϕ produce a patchier variable with well-defined clusters. Low values produce a more homogeneous spatial field.

Generation of count data

First let's generate a random covariate which will be used to define the abundance of some animal or plant species. Let's call this covariate elevation.

```
elev <- raster(matrix(rnorm(n), 50, 50), xmn = 0, xmx = 50, ymn =  
0, ymx = 50)  
plot(elev, main = "Elevation")
```



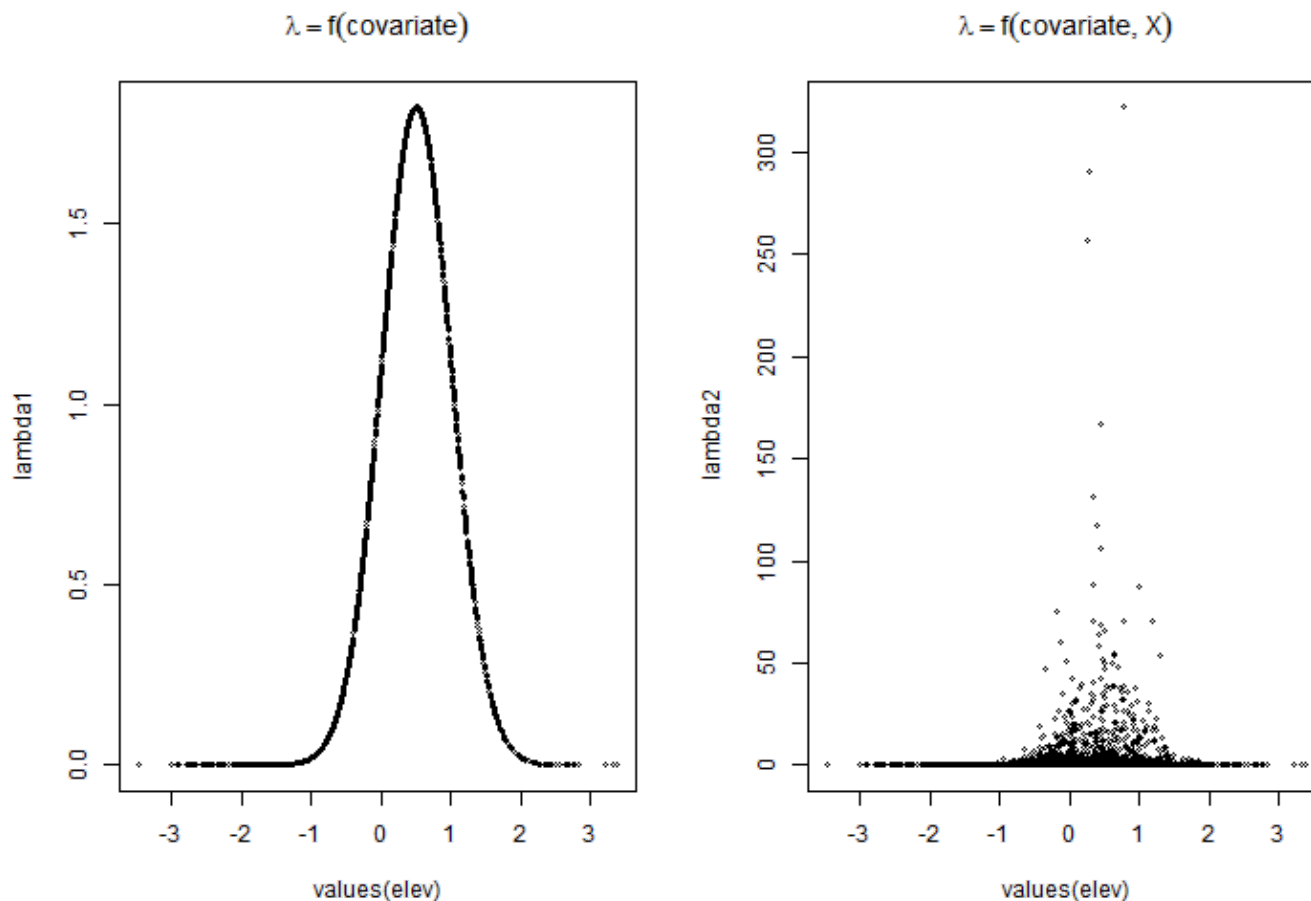
We will now simulate the abundance distribution of our study species. We choose a quadratic effect of elevation and compute the expected abundance values. We also compute a second set of expected values by including an effect of our simulated spatially autocorrelated covariate.

```
# Define coefficients of the abundance-elevation regression
beta0 <- 0.1
beta1 <- 2
beta2 <- -2

# Abundance as a quadratic function of elevation
lambda1 <- exp(beta0 + beta1 * values(elev) + beta2 *
values(elev)^2)

# Abundance as a quadratic function of elevation (+ the other
spatially
# autocorrelated covariate)
lambda2 <- exp(beta0 + beta1 * values(elev) + beta2 *
values(elev)^2 + 2 * values(Xraster))

# Plot the results
par(mfrow = c(1, 2))
plot(values(elev), lambda1, cex = 0.5, main = expression(lambda ==
f(covariate)))
plot(values(elev), lambda2, cex = 0.5, main = expression(lambda ==
f(covariate,
x)))
```

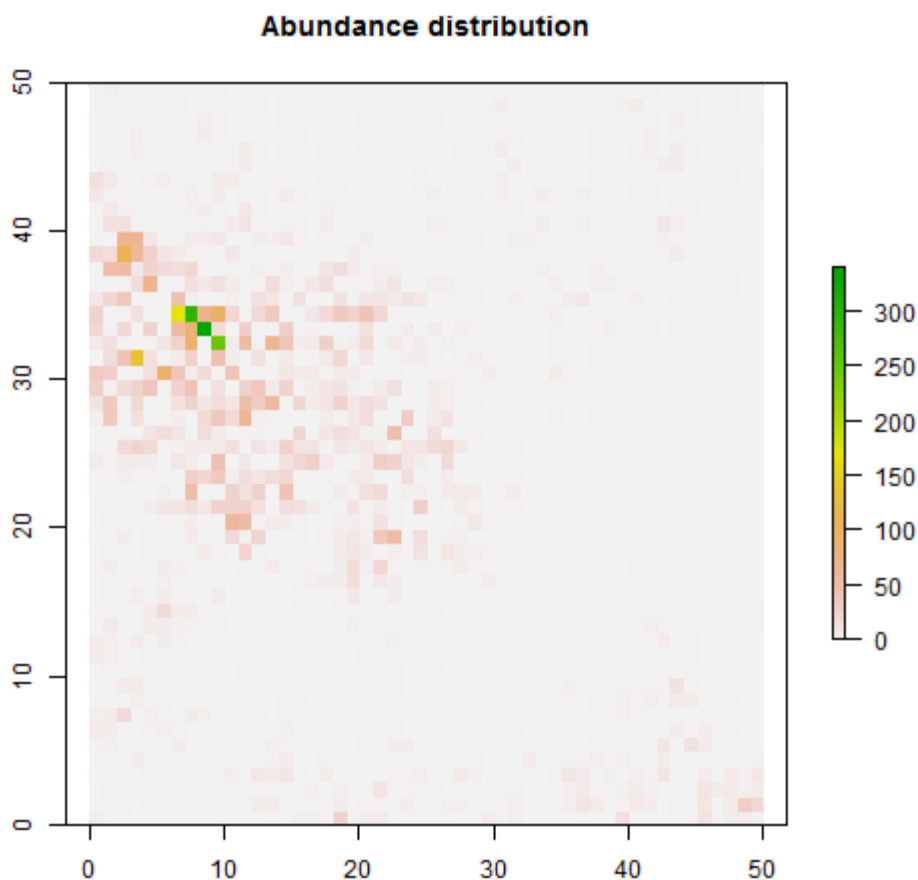


The points of the second response curve are much more dispersed around the quadratic effect of elevation. This is of course expected because the expected abundances are not only modelled as a function of elevation, but also of our simulated covariate.

We can now compute the actual counts using the second set of expected values. This is what we would observe if we would visit every pixel and count all individuals from that species (and assuming we don't miss any individual, i.e. detection probability = 1). That is, we observe a count which is at least in part “unpredictable”, or “random”, and the randomness in the counts is often assumed to be well described by a Poisson distribution with a conditional mean equal to the expected abundance just computed.

```
# Determine the actual counts
counts <- rpois(n, lambda2)
counts <- rasterFromXYZ(cbind(coordinates(elev), counts))

plot(counts, main = "Abundance distribution")
```



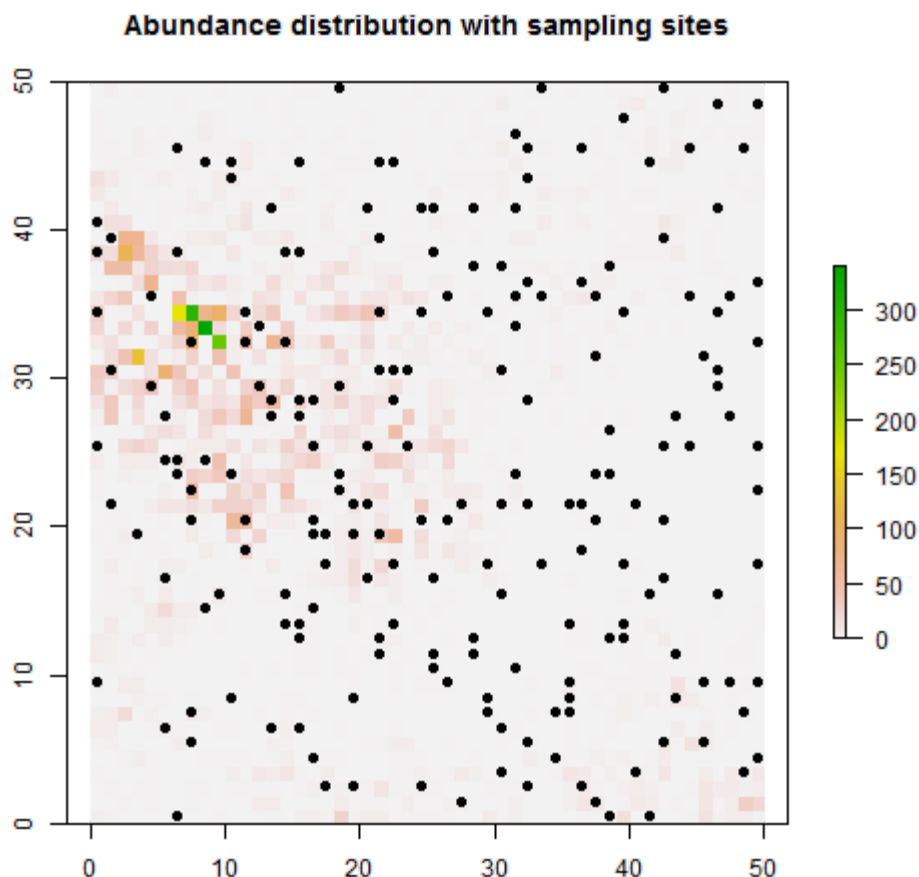
Let's imagine we visited 200 sites among the 2500 in the entire grid which represents our study area and count the birds which are present at these sites. We thus randomly choose 200 pixels and extract the related abundance values. We will then use this dataset for our subsequent analyses.

```
id <- sample(1:n, 200)
coords <- coordinates(counts)[id, ]

dat <- data.frame(coords, elev = extract(elev, coords), xvar =
extract(Xraster,
  coords), counts = extract(counts, coords))
head(dat)
```

```
##      x      y      elev      xvar counts
## 1 31.5 41.5 0.78986  0.00910      1
## 2 39.5 13.5 0.01161 -0.08804      1
## 3 14.5 32.5 0.92018  1.69372     39
## 4 21.5 11.5 1.40554 -0.52626      0
## 5 32.5 28.5 1.29872 -0.59008      0
## 6 42.5 25.5 0.29013 -1.81339      0
```

```
plot(counts, main = "Abundance distribution with sampling sites")
points(coords, pch = 16)
```



When studying spatial autocorrelation, it's better to have a fully random sampling instead of a regular one. As we will see later in this chapter, some tools used to detect and model spatial autocorrelation work better if we have some sites which are really close to one another (which

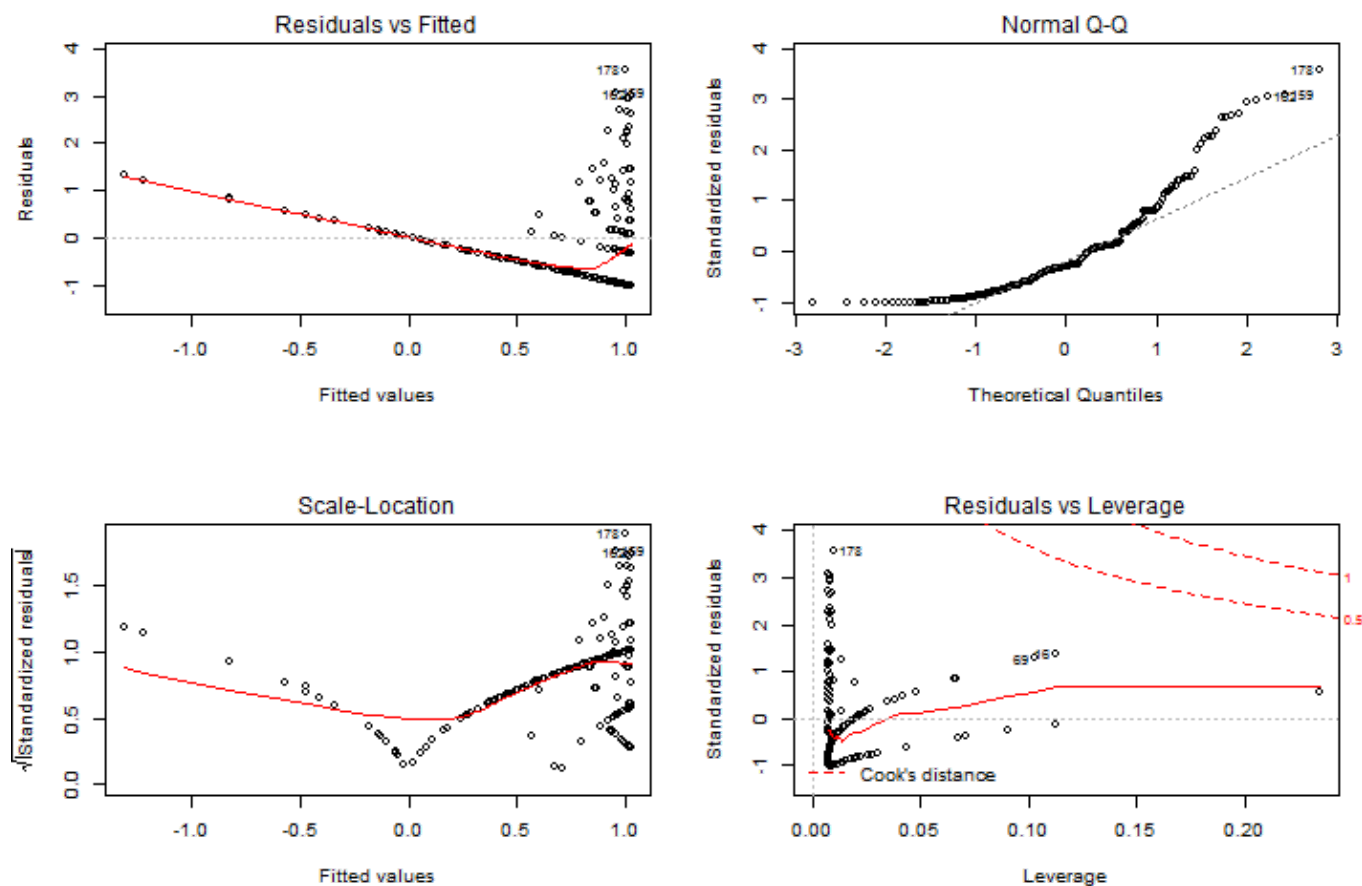
wouldn't happen with a regular sampling design). Actually a form of cluster sample or similar would give the best ability to estimate parameters of the autocorrelation function, by choosing some pairs of quadrats on purpose very close to each other.

Data analysis

We will now try to model the counts as a quadratic function of elevation. We ignore the other covariate on purpose. This omission will induce some pattern in the residuals, specifically residual spatial autocorrelation. For this example we keep everything simple and use a normal linear model with a log-transformation of the response variable (e.g. the counts). Of course it would be much better to use a Poisson GLM or a similar model (see O'Hara & Kotze, 2010). We use the following model:

$$\log(y_i + 1) = f(\text{elev}_i) + \epsilon_i \quad \epsilon_i \sim N(0, \sigma^2)$$

```
m1 <- lm(log1p(counts) ~ elev + I(elev^2), data = dat)
par(mfrow = c(2, 2))
plot(m1)
```



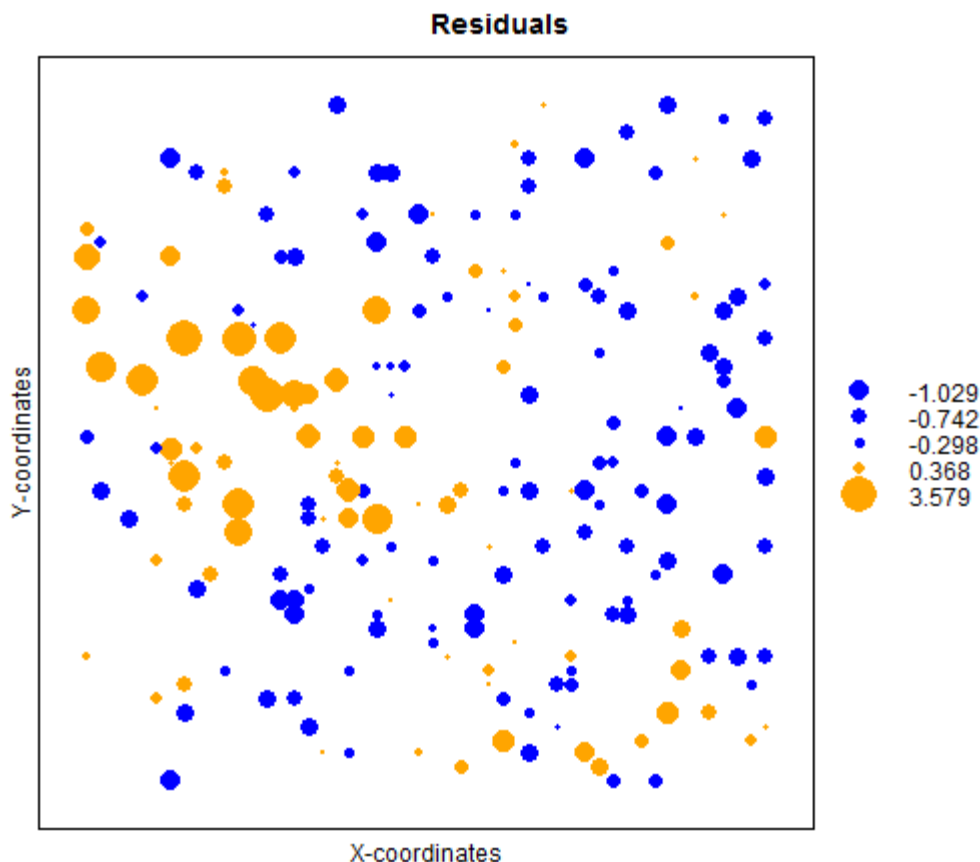
If we look at the residuals, we see a strong trend in the “residuals vs. fitted” plot (note that the “layers” in the plot are OK; they are simply a result of the discrete data). The QQ-plot is also far from ideal, showing that our residuals are not at all normally distributed (which is an assumption

of the linear model we used). We can thus conclude that our model is not appropriate for the dataset and we should not make any inference based on it.

The package *gstat* provides a nice informal diagnostic tool called a **bubble plot**. It plots the residuals versus their spatial coordinates. The size of the dots is proportional to the value of the residuals, and different colours are used for positive and negative residuals. If the independence assumption made in the standard regression model is met, then the bubble plot will not show any pattern: the sizes and the colours of the dots will be randomly distributed.

```
# Extract the standardized residuals
rs <- rstandard(m1)
# Create a spatial object (SpatialPointsDataFrame)
spdata <- data.frame(resid = rs, x = dat$x, y = dat$y)
coordinates(spdata) <- c("x", "y")

bubble(spdata, "resid", col = c("blue", "orange"), main =
"Residuals", xlab = "X-coordinates",
ylab = "Y-coordinates")
```



In our case, we see quite a strong pattern in the geographic distribution of the residuals. Positive and negative residuals seem to be clustered. If we look at the plot carefully, we can almost see the covariate which was used to simulate the data but wasn't included in the model. This is therefore a nice example that a missing covariate will induce residual spatial autocorrelation.

If you need a more formal way of detecting residual spatial autocorrelation, you can use a **spatial correlogram**. Ecologists use this tool with all sort of spatial data: geostatistical data, regular lattices, areal data. However statisticians traditionally use it only with areal data, for example to model a variable available for a set of adjacent countries.

A spatial correlogram is a plot of a statistic called **Moran's I** as a function of distance. Moran's I is computed with following formula:

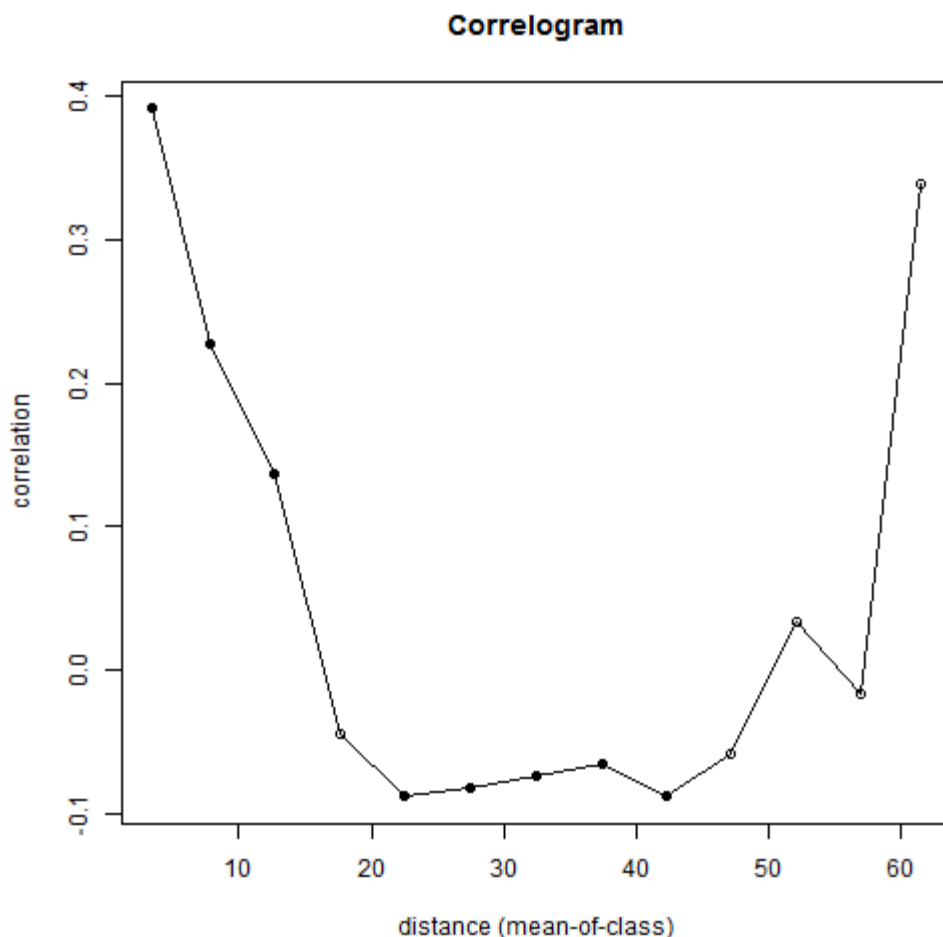
$$I = \frac{n}{w_{++}} \frac{\sum_{i=1}^n \sum_{j=1}^n w_{ij} (y_i - \bar{y})(y_j - \bar{y})}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where n equals the total number of sites in the study area, y_i is the value of the variable at site i , $\bar{y}$ is the mean value of the variable, w_{ij} represents the spatial weights, and w_{++} is the sum of all weights in the study area. In the simplest case, the w_{ij} 's have the values 1 when sites i and j are neighbours or 0 when they're not neighbours. If there is no spatial autocorrelation, the expected value of I is $E(I) = -1/(n-1)$. This value is close to 0 when n is large. If the computed I is greater than $-1/(n-1)$, then the variable is spatially clustered. The null hypothesis of no spatial correlation can be tested using a normal approximation or a permutation test.

As you can see, the way we define the weights w_{ij} 's has a strong influence on the actual value of I . One possibility of defining the neighbours ($w_{ij} = 1$) is to use a distance threshold, for example two sites are considered as neighbours if the distance between them is smaller than or equal to some distance, such as 2 km. It is of course possible to define a range of thresholds. For example we first compute Moran's I considering neighbours between 0 and 2 km, then between 2 and 4 km, etc. If we plot these values of I as a function of distance, we get a spatial correlogram. To produce a meaningful correlogram, the width of the distance bands should be greater than the minimum inter-site distance in our study area.

The *correlog* function of the *ncf* package allows computing a spatial correlogram, the significance of the different I values is tested with permutation. The packages *pgirmess* and *spdep* also have functions to compute a spatial correlogram.

```
fit1 <- correlog(x = spdata$x, y = spdata$y, z = spdata$resid,
  increment = 5,
  resamp = 100, quiet = TRUE)
plot(fit1)
```



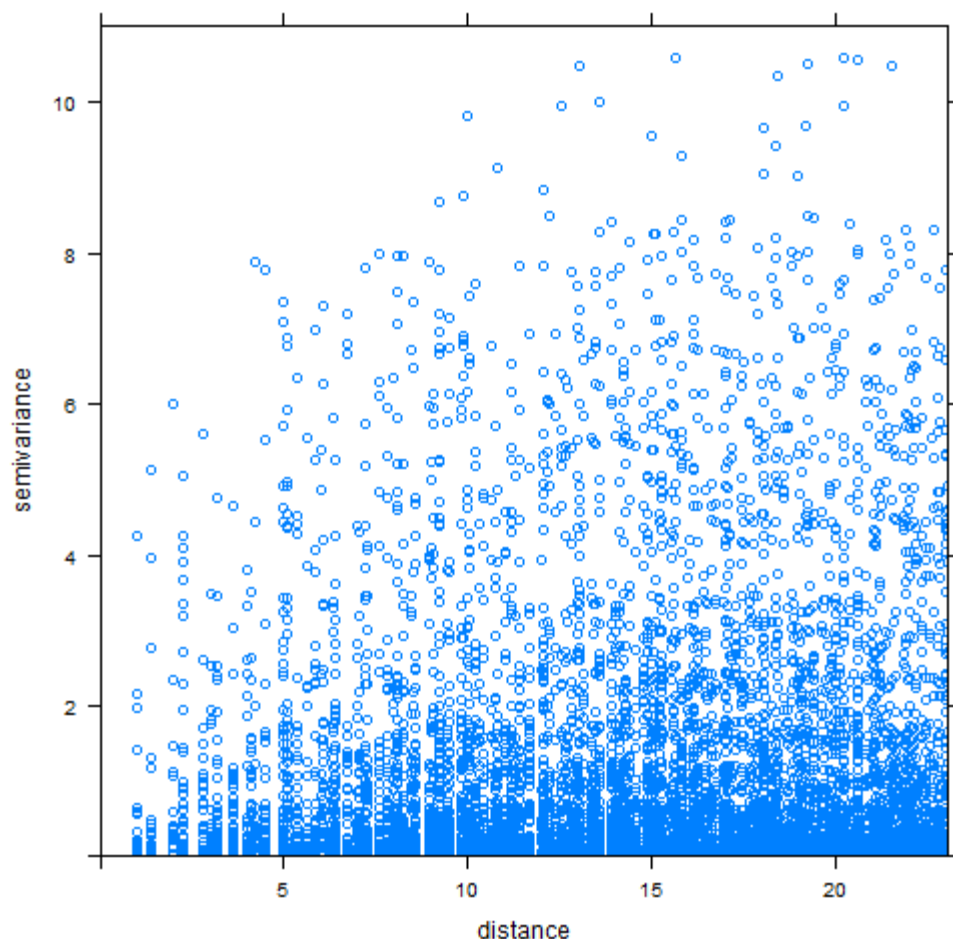
The black dots indicate values of $|r|$ significantly larger or smaller than expected under the Null hypothesis of no autocorrelation, i.e. than $\pm 1/(n-1)$. This plot shows quite a strong spatial autocorrelation in the residuals for the first distance classes. Starting with a distance of 15, the $\log(\text{counts})$ appear to be more or less independent. The significant negative autocorrelation on the rest of the plot is hard to interpret (and because it is really close to 0, it can probably be ignored).

Most statisticians working with geostatistical data use another tool called a **variogram** (= semi-variogram) to detect residual spatial autocorrelation. The variogram is a plot of the semi-variance as a function of distance. As a first approach, we can plot the semi-variance as a function of distance for every possible pair of sites. This is called a **variogram cloud**. The semi-variance between two sites is defined by:

$$\gamma(x_1, x_2) = \frac{1}{2} E[(Z(x_1) - Z(x_2))^2]$$

where (x_1) and (x_2) are the coordinates of the site and (Z) is a random function modelling the phenomenon under study. We are going to use this tool on our residuals to detect spatial autocorrelation. The *gstat* package has some really useful functions for variogram analyses.

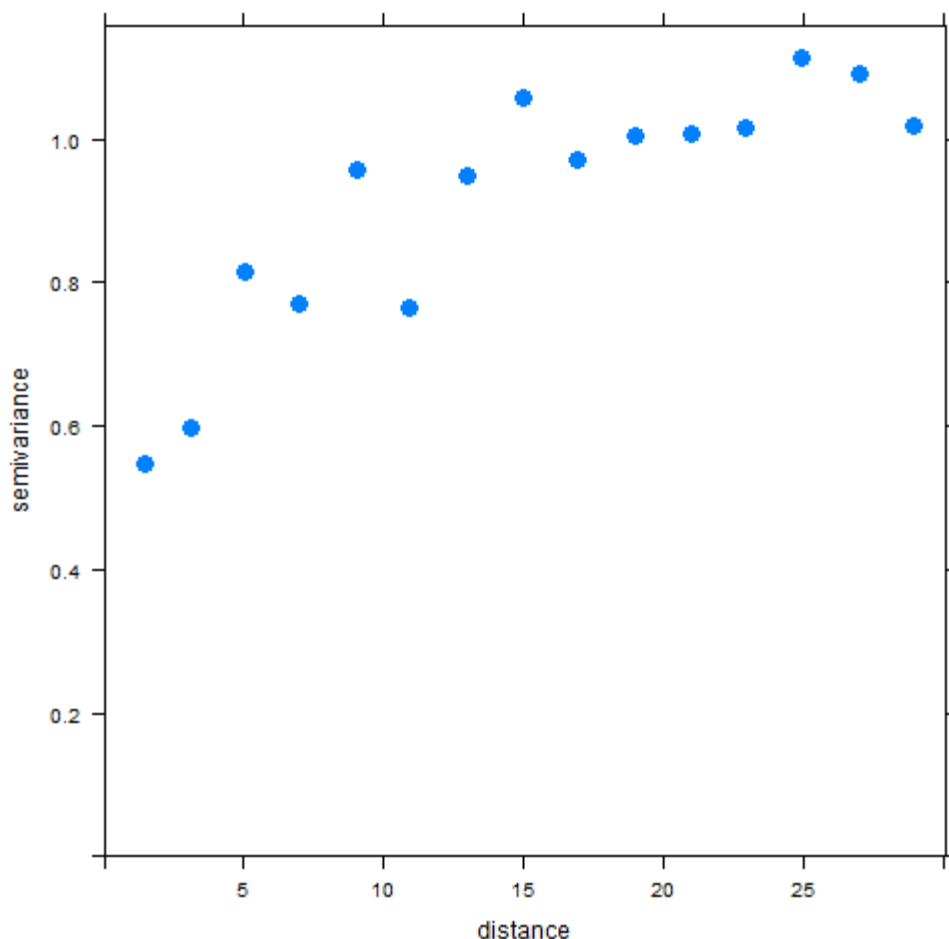
```
variocloud <- variogram(resid ~ 1, spdata, cloud = TRUE)
plot(variocloud)
```



This plot is rather difficult to interpret due to the massive density of plotting symbols. However, it seems that the semi-variance is increasing with distance. This means that sites which are close to each other are more similar than sites which are more distant. This is the exact definition of positive spatial autocorrelation!

To improve readability, the distances are often grouped into distance classes, and an average value (the mean or some other robust estimator) of the semi-variances is displayed on the plot for each distance class. This is called a **sample** (or experimental) **variogram**, and it is probably one of the most useful (and used) tools in geostatistics. Note that the standard variogram assumes isotropy (just like the spatial autocorrelogram), which means that spatial dependence of the residuals is the same in any direction over the whole study area (i.e. there is no directional dependence of the correlation structure). Otherwise we have to use a directional variogram (see *?variogram*). The *cutoff* parameter is the maximum distance up to which point pairs are considered. Autocorrelation should not be computed for very distant pairs because the semi-variance usually shows wild oscillations which are essentially meaningless.

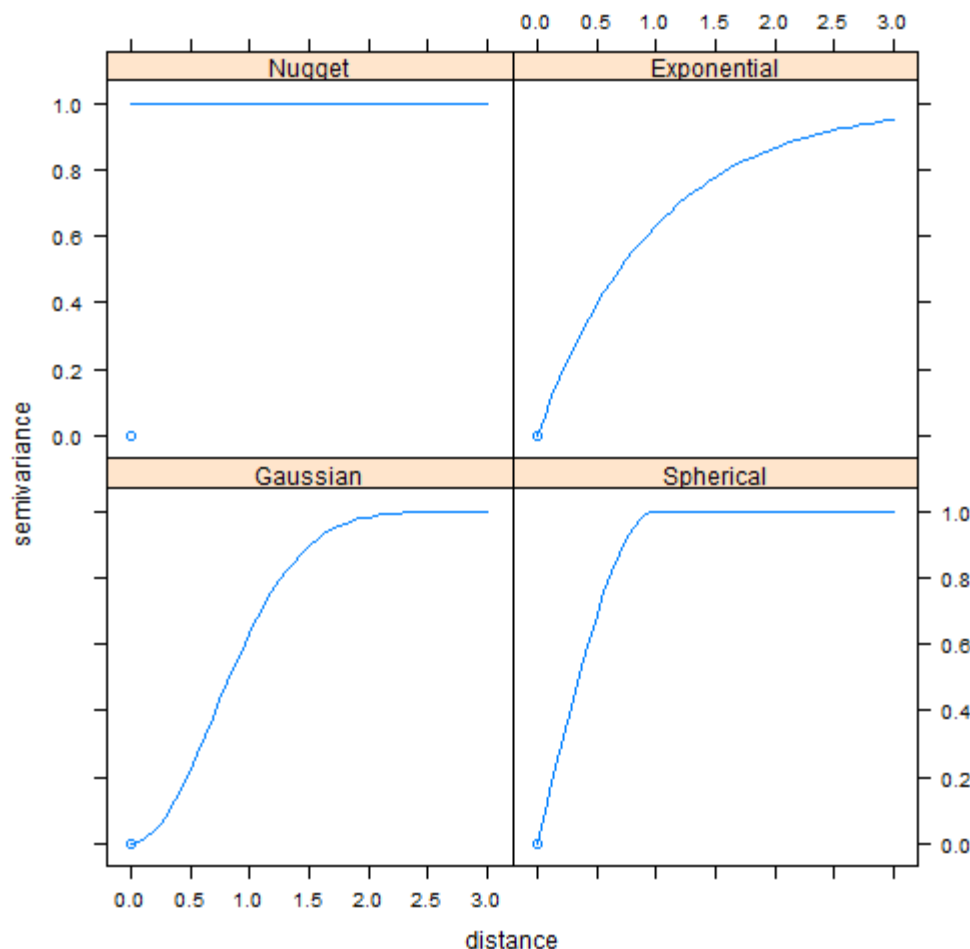
```
vario <- variogram(resid ~ 1, spdata, cutoff = 30)
plot(vario, pch = 16, cex = 1.5)
```



This confirms our first impressions: the semi-variance is first increasing with distance and seems to reach some plateau for longer distance. We are now convinced that spatial autocorrelation is present in our residuals. The sample variogram is always a combination of the true (latent) variogram and some sampling error. If we want to be sure that an increase in semi-variance is not simply due to chance, we can test it using permutations. We need to compute variograms from the same data but at every iteration we randomly reassign measurements to the spatial locations, thus breaking the correlation with distance between locations. If our sample variogram is within the range of these replicated random variograms, then our increase in semi-variance may due to chance alone.

The best solution for dealing with residual spatial autocorrelation in our model would be to find the missing covariate and to include it in the model. Unfortunately this is not always possible (the covariate is not known or not available, or is too expensive to acquire) and we have to find another solution. As is typical with violations of modelling assumptions, one solution is always to model the offending pattern. That is, we can model this residual spatial autocorrelation and include this model in our linear model of abundance.

To do that, we need to fit a theoretical model to our sample variogram. The most common variogram models used by ecologists are the nugget, the exponential, the gaussian and the spherical models. They can be fit easily in R using the packages *gstat* or *nlme*. They look like this:



A standard variogram is usually described by 3 parameters. The range is the distance at which the sites are no more autocorrelated (the start of the plateau). The sill is the semi-variance value corresponding to the range. And lastly, the nugget is the intercept of the variogram. It represents the unexplained variation caused by spatial structure at distance less than the minimum lag or by measurement error.

Exercise

Using the *Fulmaris glacialis* dataset (RIKZ, 1999) included in the *gstat* package, try to model the counts as a function of the sea water depth and/or the distance to the coast. Check if you have any spatial autocorrelation in your residuals.

```
data(fulmar)
str(fulmar)
```

```
## 'data.frame':    1324 obs. of  6 variables:
## $ year   : num  1998 1998 1998 1998 1998 ...
## $ x      : num  614192 613151 619645 608859 556583 ...
## $ y      : num  5875490 5872947 5888800 5839623 5738519 ...
## $ depth  : num   6  6  9 16  1  3  3  3 15 14 ...
## $ coast  : num   3.45 3.45 3.04 3.85 2.7 ...
## $ fulmar: num    0  0  0  0  0  0  0  0  0  0 ...
```